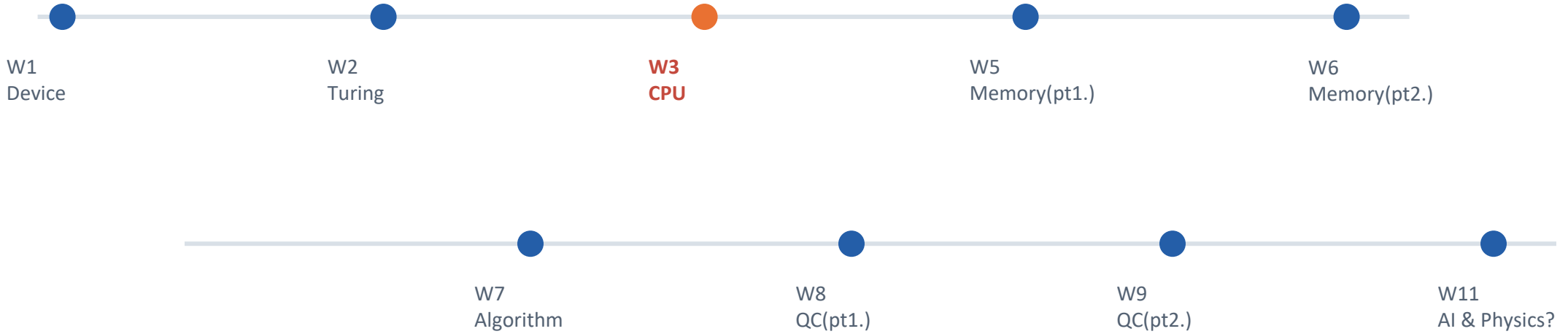
CPU Architecture and Data Movement (RISC-V)

Jaehyun Choi
Department of Computer Science and Engineering
Seoul National University

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Memory(pt2.)
W7
Algorithm
W8
QC(pt1.)
W9
QC(pt2.)
W11
AI & Physics?


CPU Components
Single Cycle CPU
Pipelined CPU

Key Question

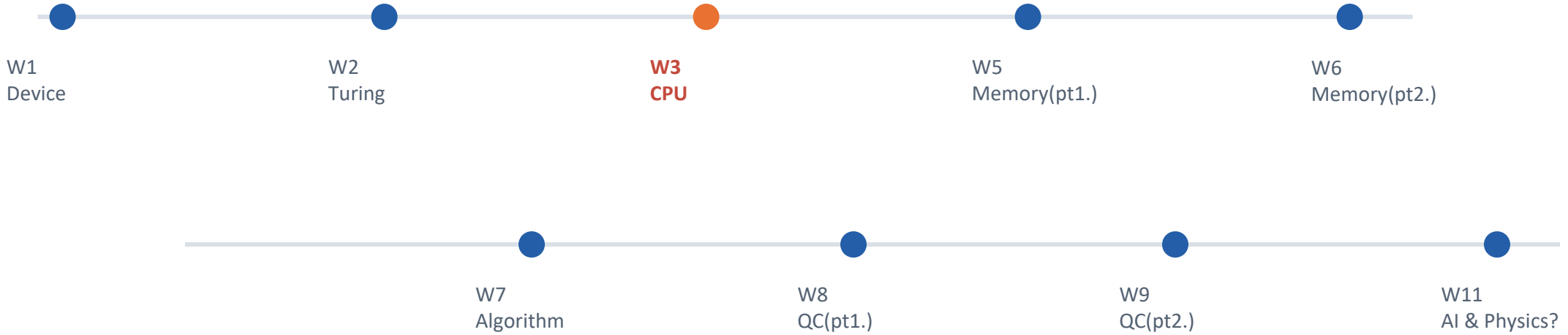
- How is a single line of code executed inside a computer?

```
</> C
a[2] = a[0] + a[1];
```

- Program execution is controlled data movement.

- Memory ↔ Registers ↔ ALU

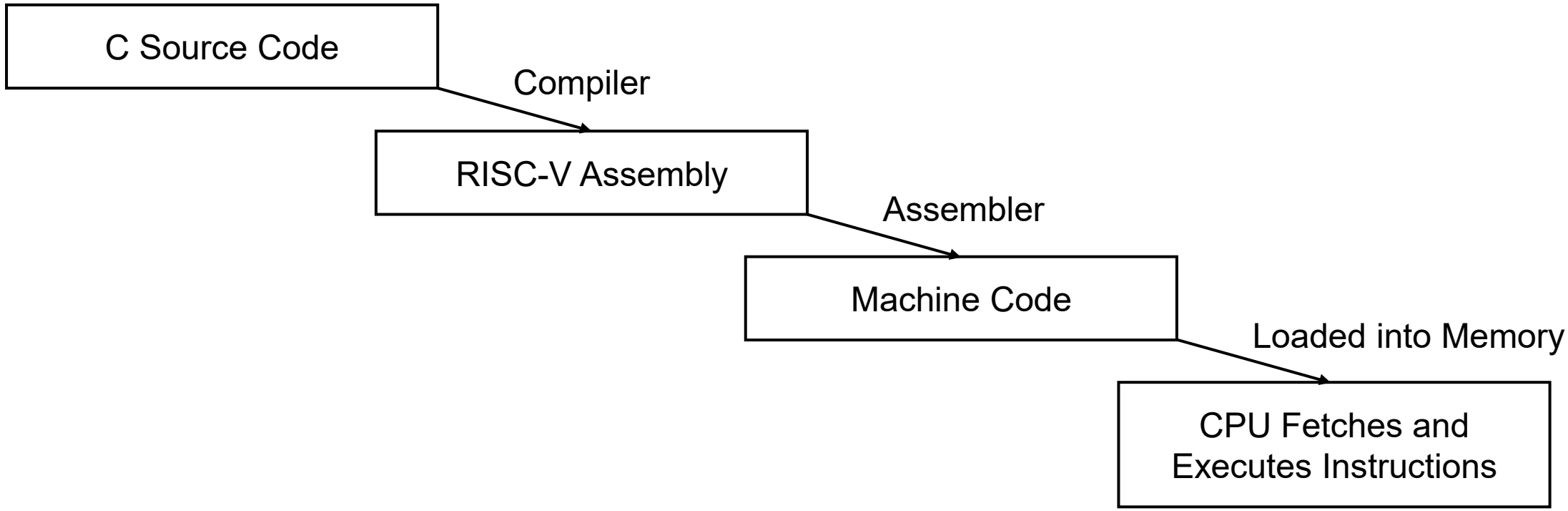
RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Memory(pt2.)
W7
Algorithm
W8
QC(pt1.)
W9
QC(pt2.)
W11
AI & Physics?


CPU Components
Single Cycle CPU
Pipelined CPU

From Source Code to CPU Execution

- The CPU does not directly understand C code
- It executes binary instructions stored in memory



ISA and Microarchitecture

- ISA defines the interface between software and hardware

```
</> asm
add x5, x6, x7
```

```
x5 ← x6 + x7
```

- Microarchitecture defines how the ISA is implemented in hardware
  - Single-Cycle CPU
  - Pipelined CPU
- RISC-V is an ISA. A single-cycle CPU is one possible implementation of RISC-V.

Basic Principles of RISC-V

- RISC-V uses a load-store architecture.
  - Arithmetic operations are performed on registers
  - Memory is accessed only through load and store instructions

```asm
</> asm
add x5, x6, x7     # x5 = x6 + x7
lw  x8, 0(x5)      # x8 = Memory[x5 + 0]
sw  x8, 4(x5)      # Memory[x5 + 4] = x8
```

- CPU does not manipulate variable names
- It manipulates register numbers and memory addresses

RISC-V Registers

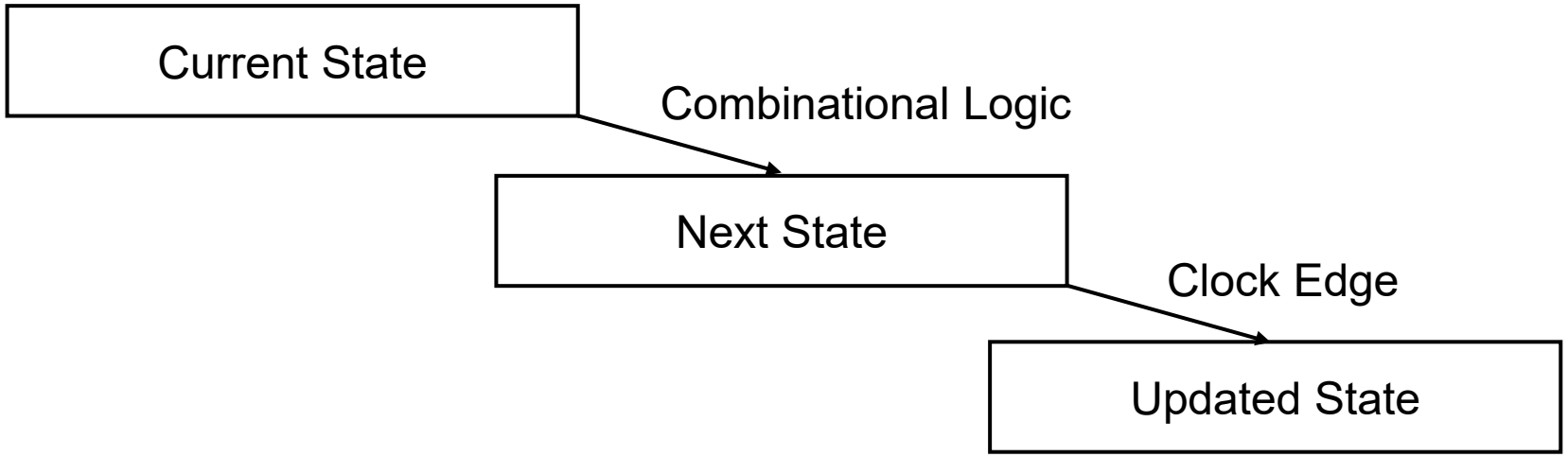
- RISC-V Registers
  - x0 – x31: 32 general-purpose registers
  - x0: always contains the value 0

Hardware Components of a CPU

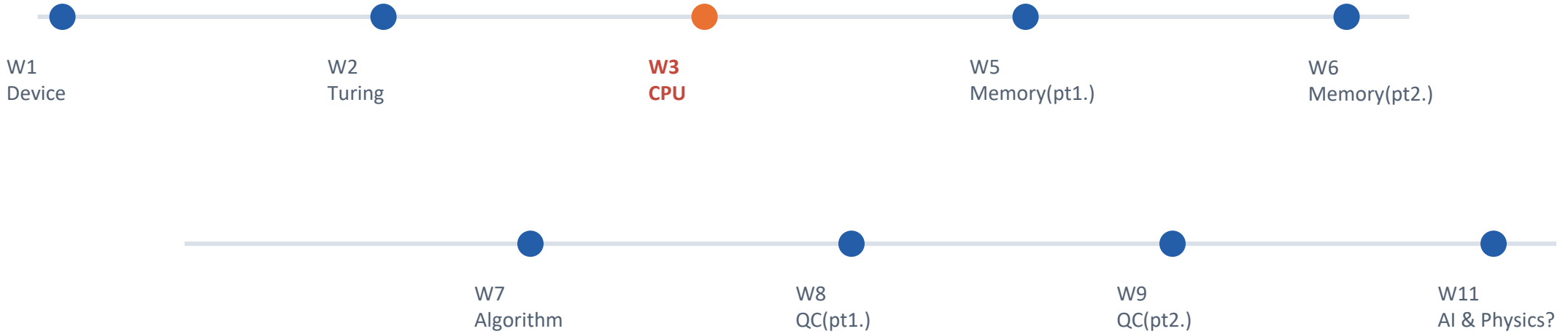
- Combinational Logic
  - The output depends only on the current inputs
  - Examples) ALU, Multiplexer, Decoder
- Sequential Logic
  - The circuit stores state across clock cycles
  - Examples) Registers, Program Counter, Memory
- Main CPU components
  - Program Counter, Instruction Memory, Register File, ALU, Data Memory, Immediate Generator, Multiplexers, Control Unit

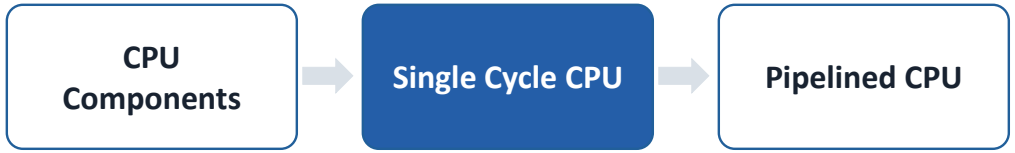
Clock and State

- During a clock cycle
  - Combinational logic computes the next values (Computation)
- At the clock edge
  - The PC, registers, and memory store the new values (State Update)



RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Memory(pt2.)
W7
Algorithm
W8
QC(pt1.)
W9
QC(pt2.)
W11
AI & Physics?


CPU Components
Single Cycle CPU
Pipelined CPU

Executing a Short Program

```
</> C
a[2] = a[0] + a[1];
```

```
</> asm
lw   x5, 0(x10)
lw   x6, 4(x10)
add  x7, x5, x6
sw   x7, 8(x10)
```

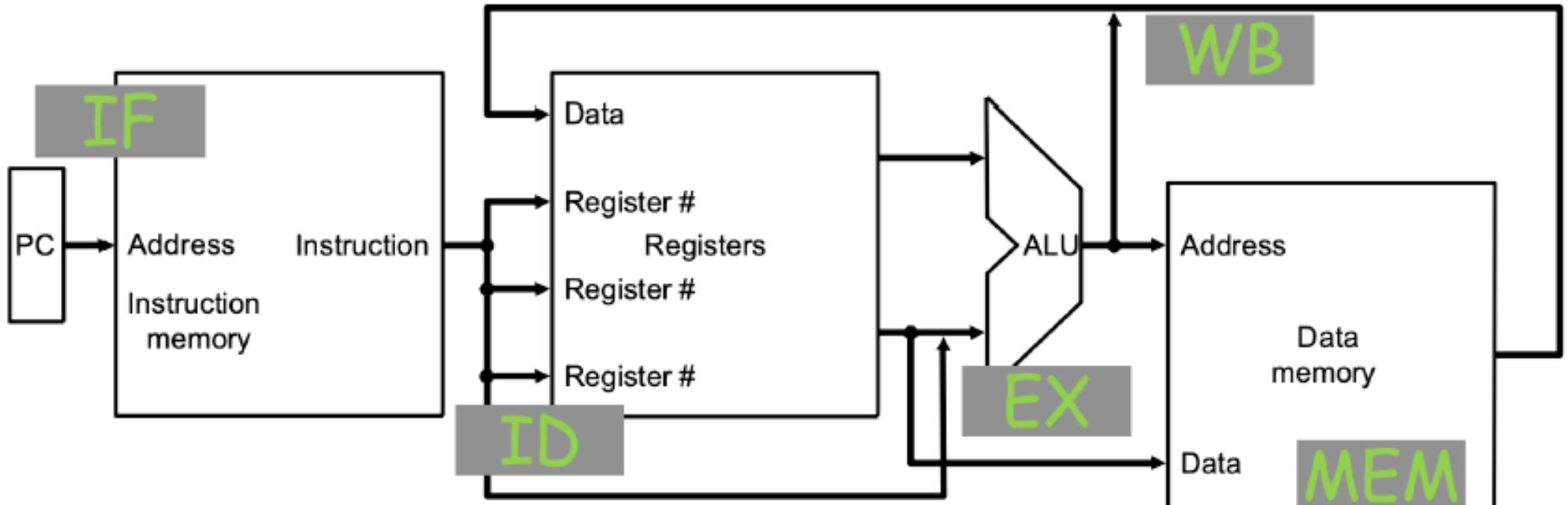
```
Initial State
x10 = 1000          #address of a
Memory[1000] = 10   #a[0]
Memory[1004] = 20   #a[1]
```

```
Execution Result
x5 = 10
x6 = 20
x7 = 30
Memory[1008] = 30   #a[2]
```

```
Data Movement
Memory → Register
Memory → Register
Register → ALU → Register
Register → Memory
```

Basic Single-Cycle Datapath

- In a single-cycle CPU, one instruction completes within one clock cycle
  - Fetch → Decode → Execute → Memory → Write Back
  - All five steps occur within the same clock cycle



**Based on figures from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Instruction Fetch

- The Program Counter stores the address of the current instruction

```
Instruction = InstructionMemory[PC]
```

- For a normal sequential instruction

```
NextPC = PC + 4
```

- Why add 4?
  - One standard RV32I instruction is 32 bits = 4 bytes
- Is the next PC always PC + 4?
  - No, Branch and jump instructions may change the control flow

RISC-V Instruction Formats

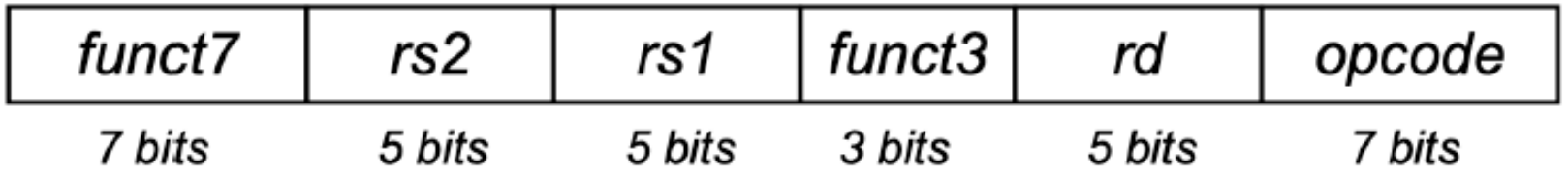
Name (Bit position)	Fields					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

R-type Example: ADD

- Assembly

  ADD rd_{reg} $rs1_{reg}$ $rs2_{reg}$

- Machine Encoding

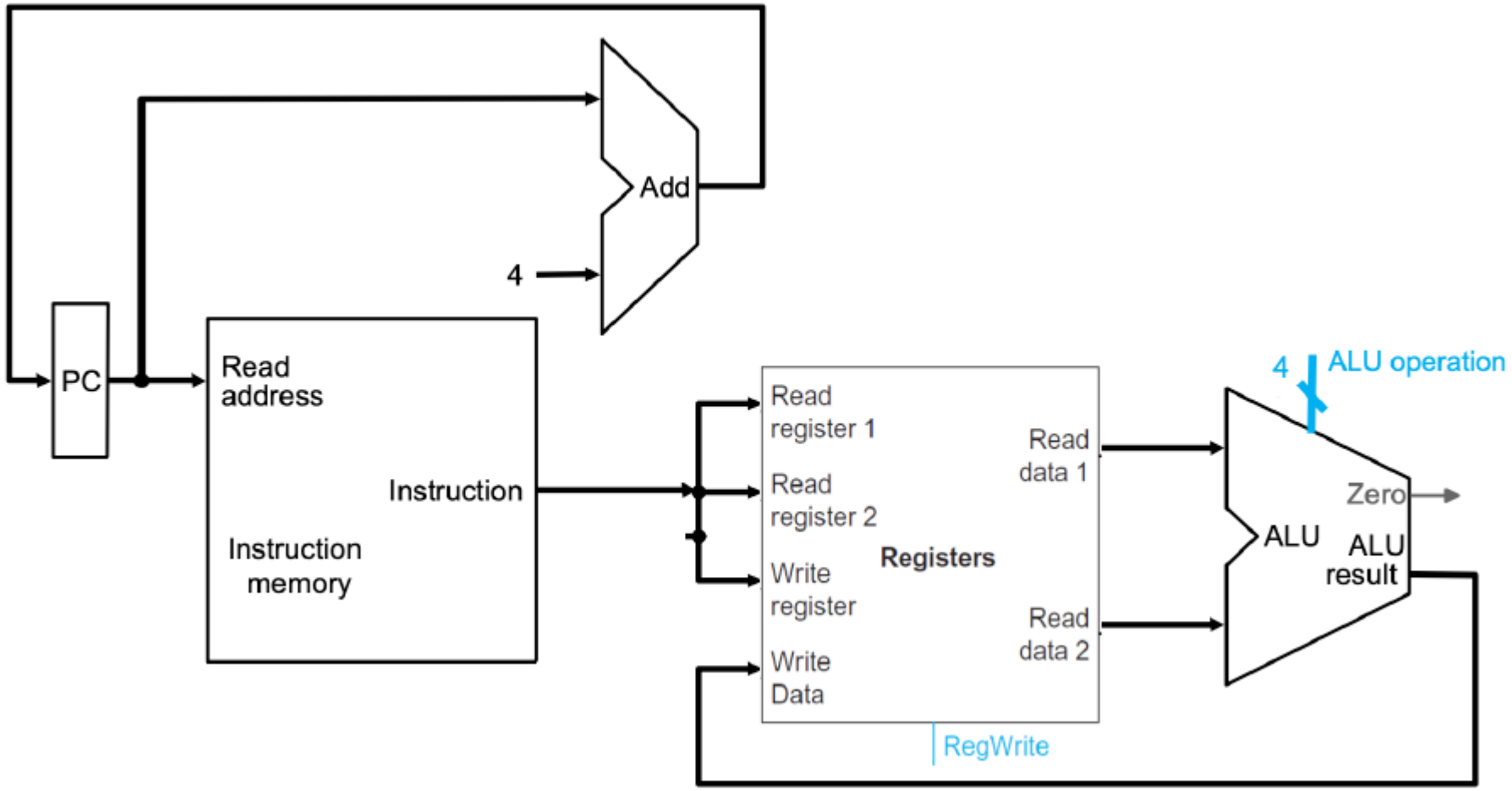


R-type

- Instruction Semantics

```
if MEM[PC] == ADD rd rs1 rs2
    GPR[rd] ← GPR[rs1] + GPR[rs2]
    PC ← PC + 4
```

Data path for R-type ALU instructions

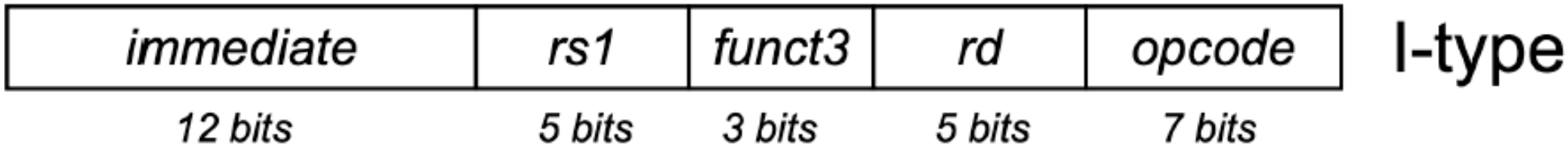


I-Type Example: ADDI

- Assembly

  ADDI rd_{reg} $rs1_{reg}$ $immediate_{12}$

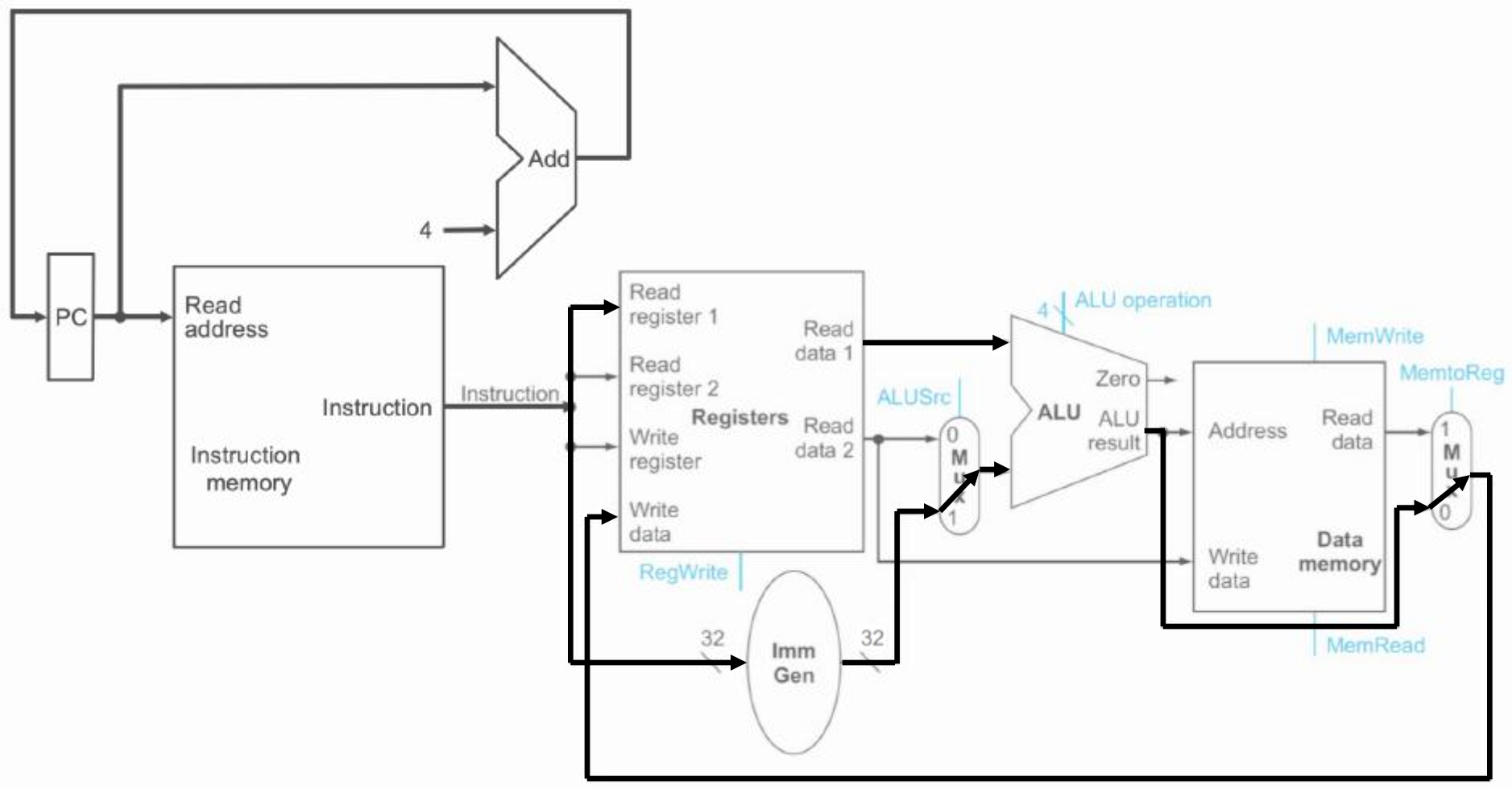
- Machine Encoding



- Instruction Semantics

```
if MEM[PC] == ADDI rd rs1 immediate
    GPR[rd] ← GPR[rs1] + sign-extend (immediate)
    PC ← PC + 4
```

Data path for I-type ALU instructions

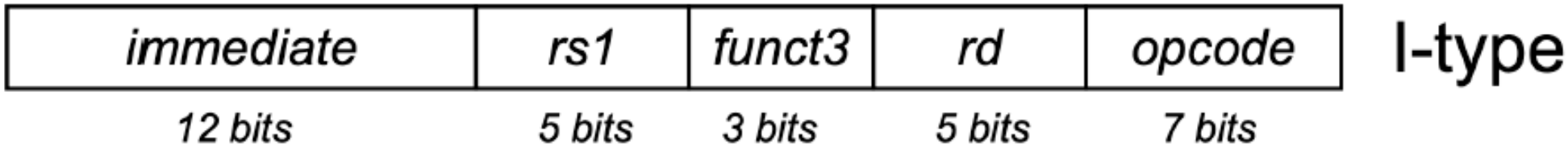


Load Example: LW

- Assembly

  LW rd_{reg} $offset_{12}$ ($rs1_{reg}$)

- Machine Encoding



- Instruction Semantics

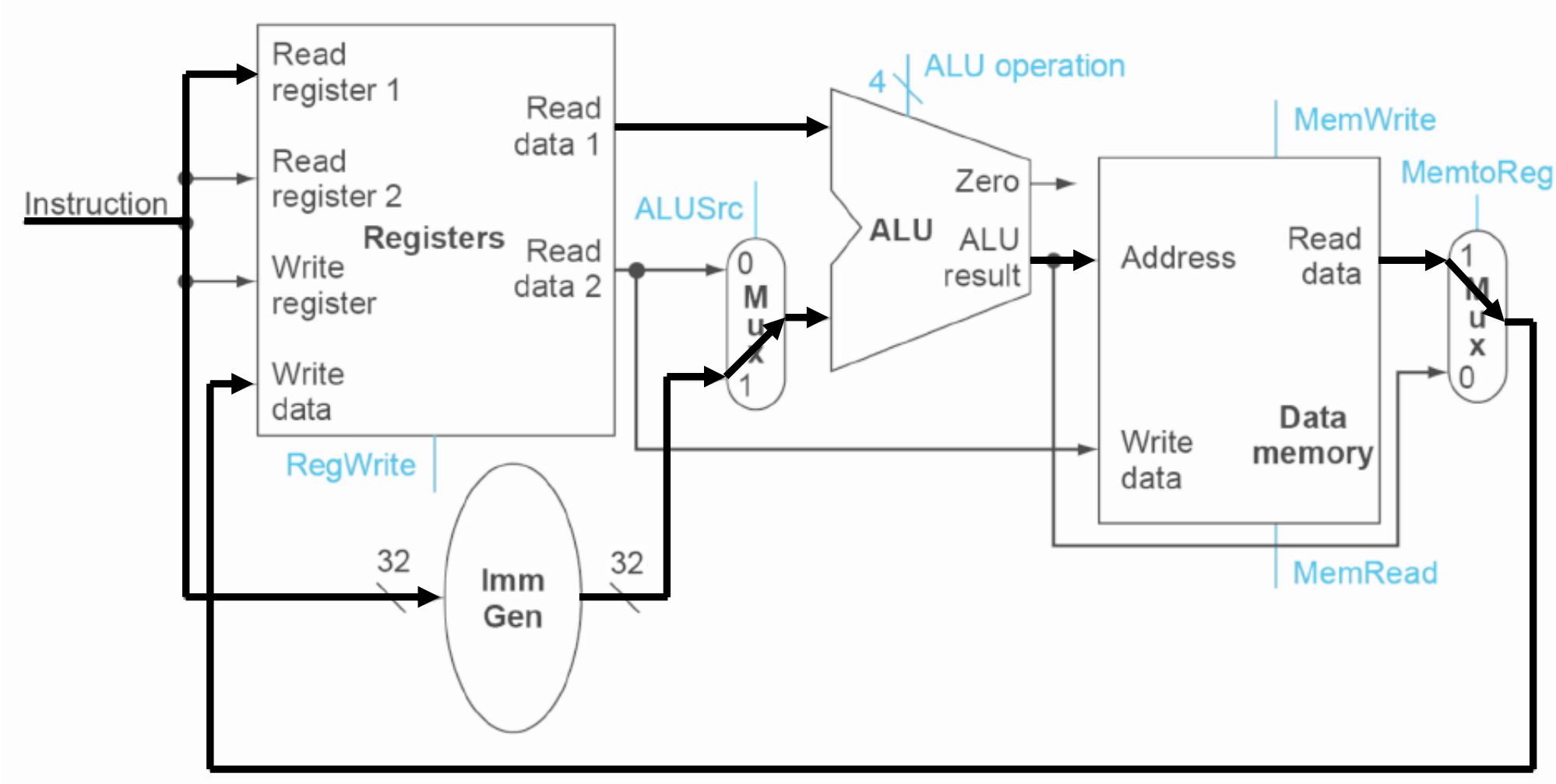
  if MEM[PC]==LW rd $offset_{12}$ (rs1)

  EA = sign-extend(offset) + GPR[rs1]

  GPR[rd] ← MEM[translate(EA)]

  PC ← PC + 4

Data path for Load Instructions

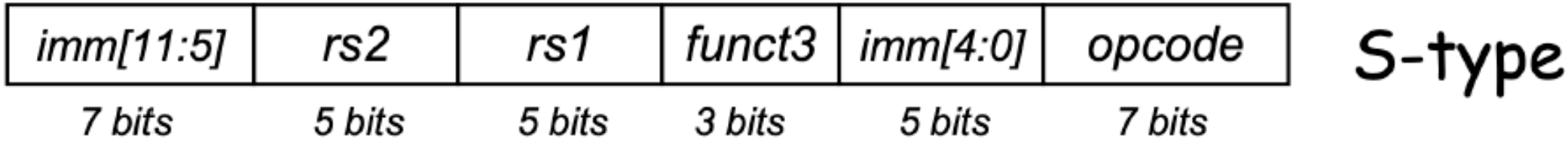


Store Example: SW

- Assembly

  SW $rs2_{reg}$ $offset_{12}$ ($rs1_{reg}$)

- Machine Encoding



S-type

- Instruction Semantics

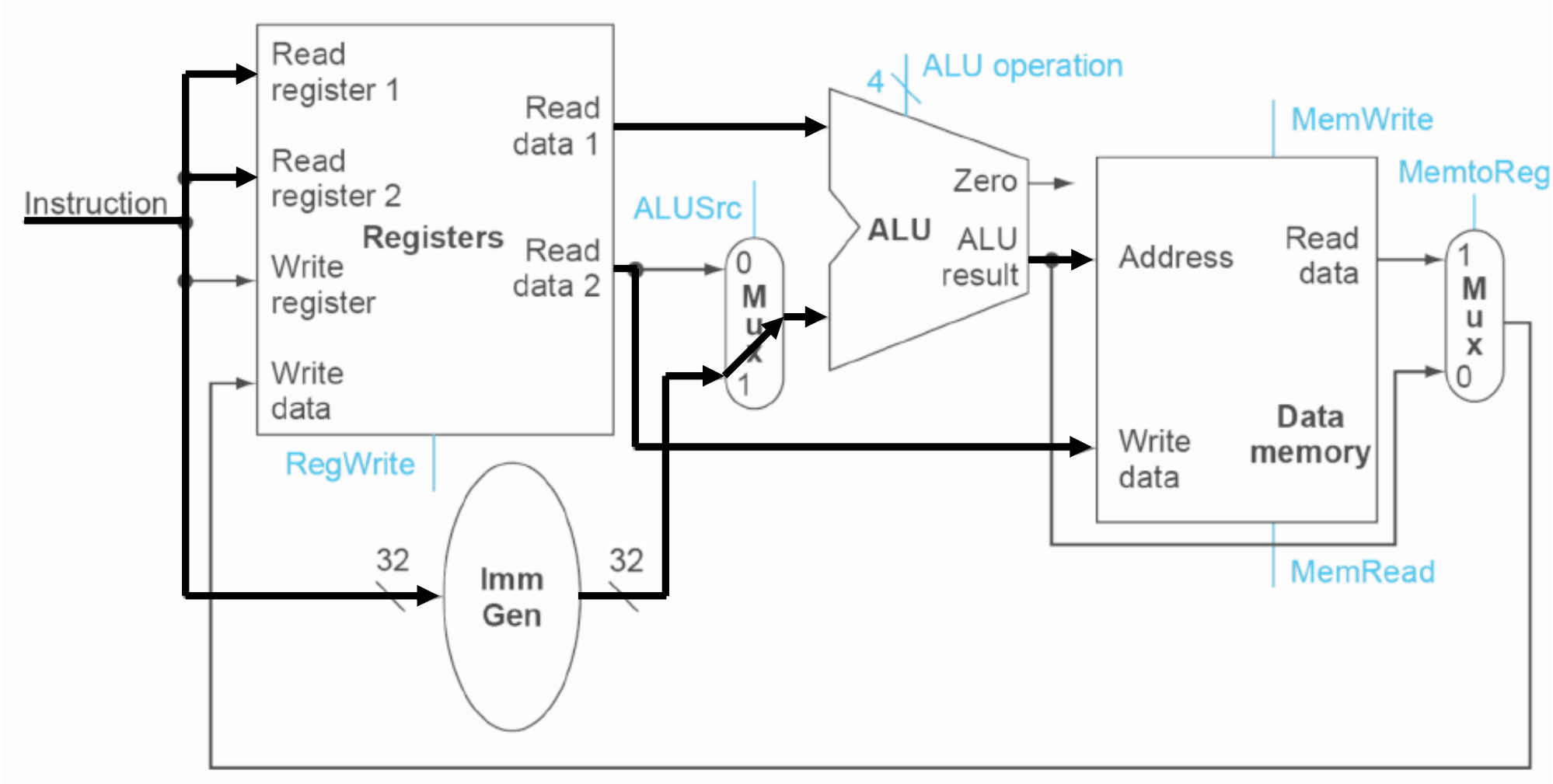
  if MEM[PC]==SW rs2 $offset_{12}$ (rs1)

  EA = sign-extend(offset) + GPR[rs1]

  MEM[translate(EA)] ← GPR[rs2]

  PC ← PC + 4

Data path for Store Instructions

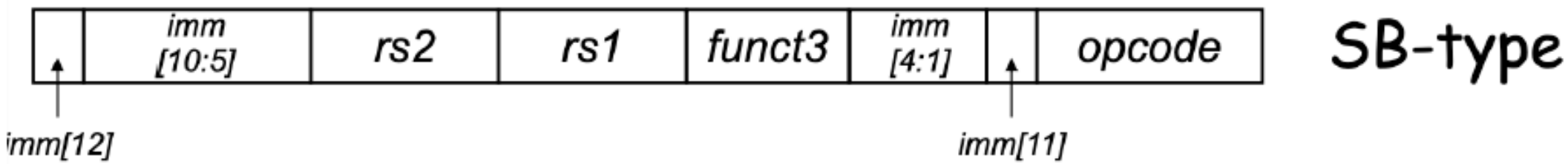


Branch Example: BEQ

- Assembly

  BEQ $rs1_{reg}$ $rs2_{reg}$ $immediate_{12}$

- Machine Encoding



- Instruction Semantics

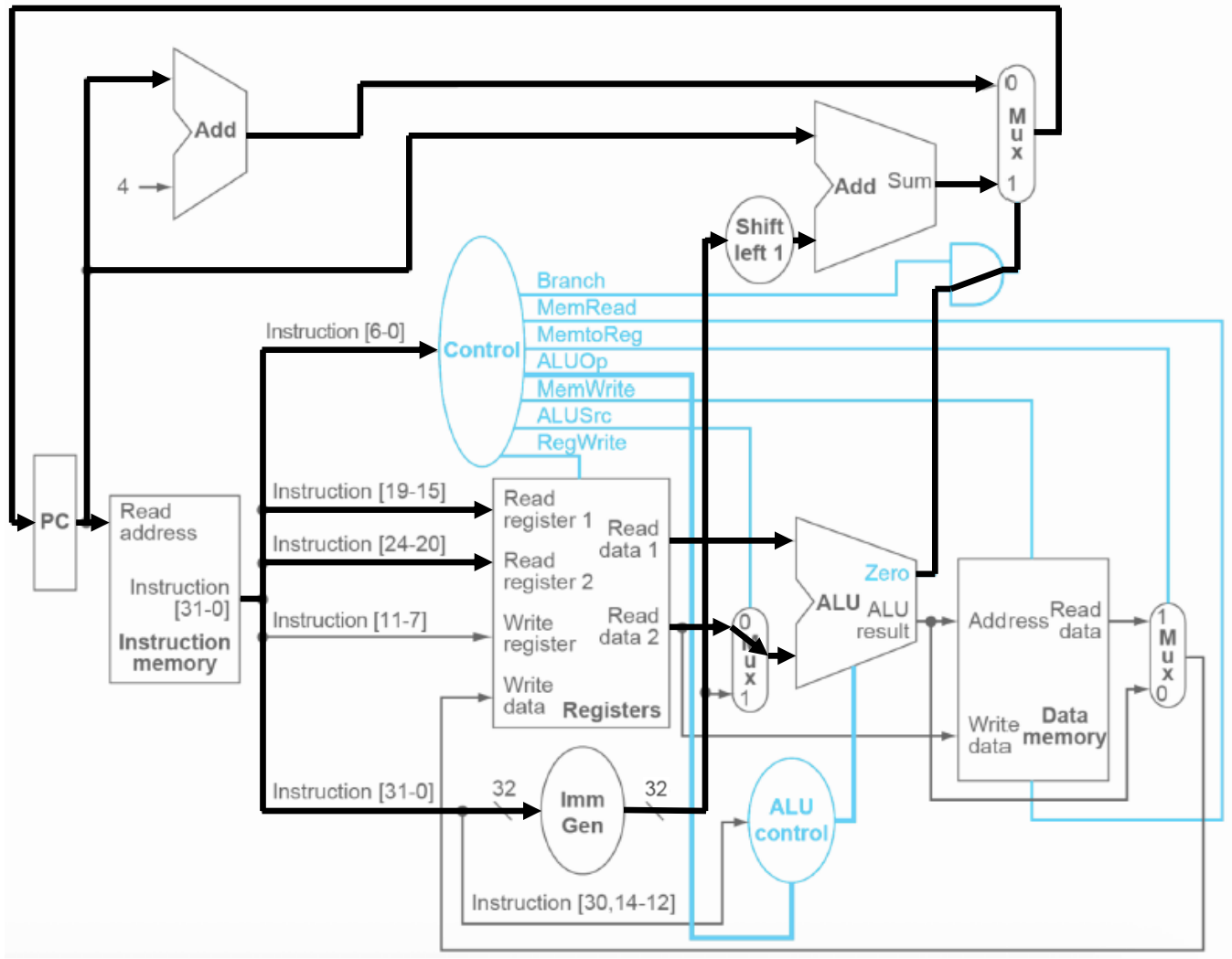
  if MEM[PC]==BEQ rs1 rs2 $immediate_{12}$

  target = PC + sign-extend(immediate) x 2

  if GPR[rs1]==GPR[rs2] then PC ← target

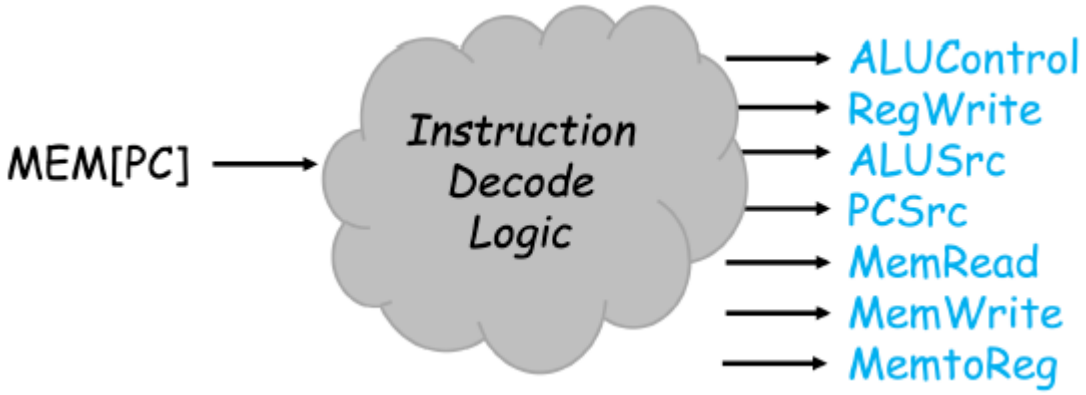
  else PC ← PC + 4

Data path for Branch Instructions



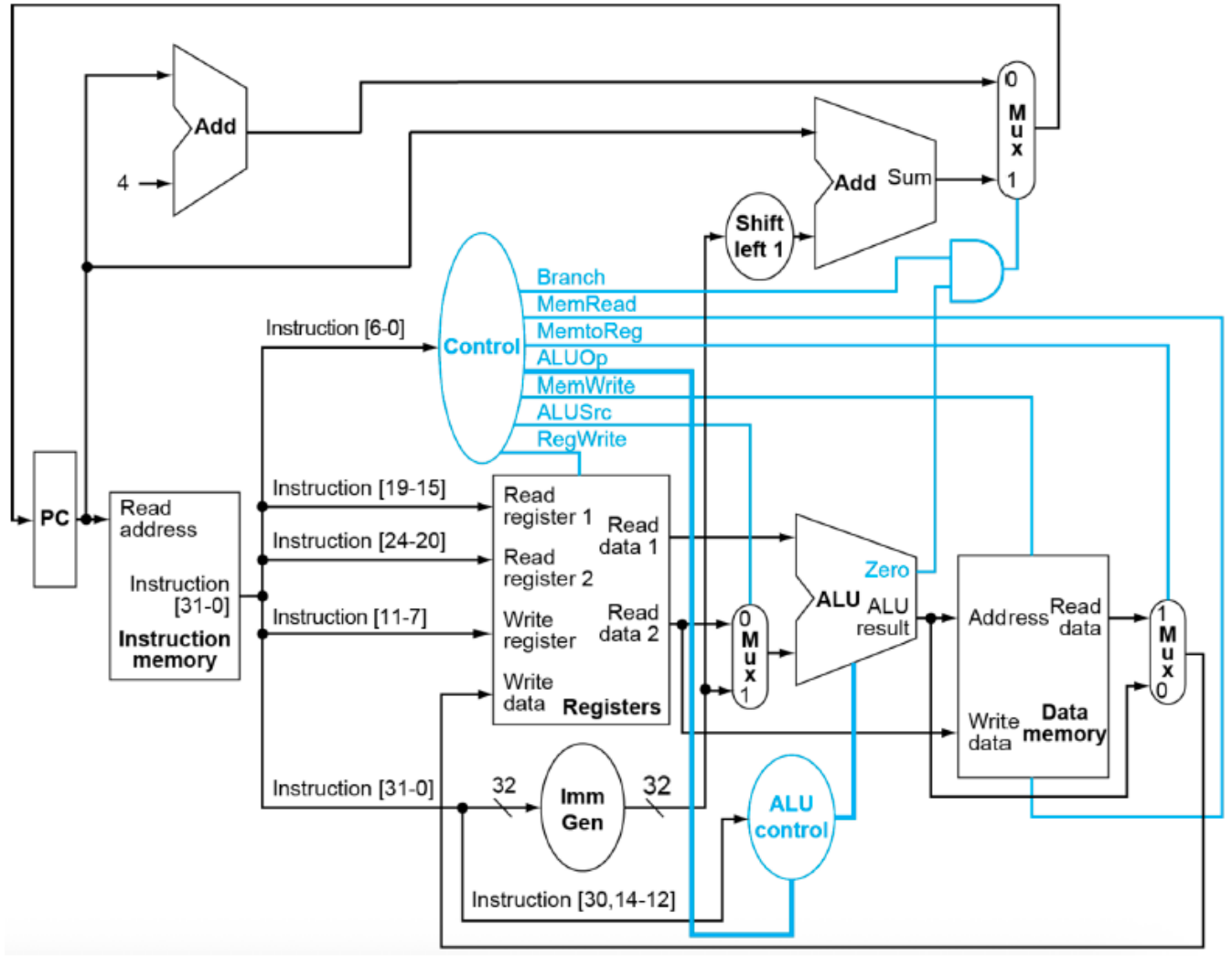
Control Unit

- Combinational logic that decodes the instruction



Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Complete Single-Cycle Datapath



Executing a Short Program

```
</> C
a[2] = a[0] + a[1];
```

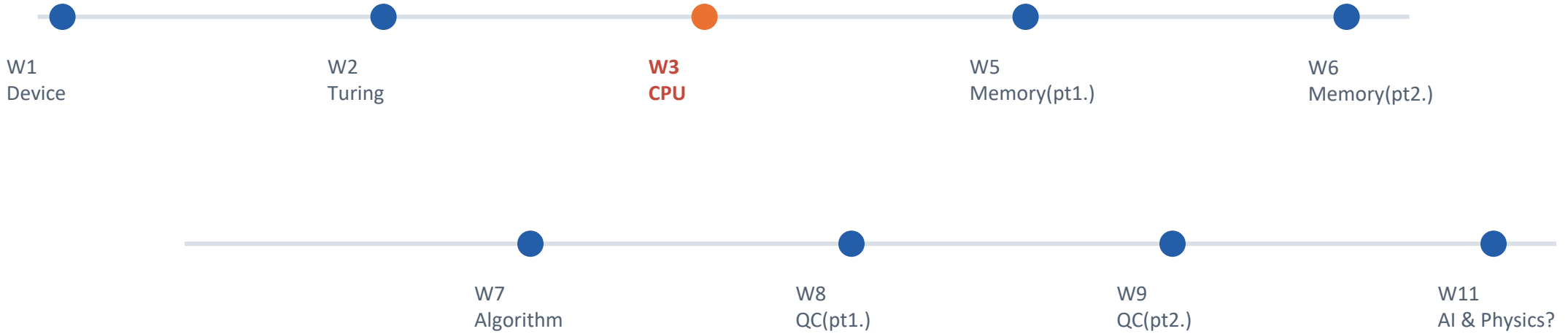
```
</> asm
lw   x5, 0(x10)
lw   x6, 4(x10)
add  x7, x5, x6
sw   x7, 8(x10)
```

```
Initial State
x10 = 1000          #address of a
Memory[1000] = 10   #a[0]
Memory[1004] = 20   #a[1]
```

```
Execution Result
x5 = 10
x6 = 20
x7 = 30
Memory[1008] = 30   #a[2]
```

```
Data Movement
Memory → Register
Memory → Register
Register → ALU → Register
Register → Memory
```

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Memory(pt2.)
W7
Algorithm
W8
QC(pt1.)
W9
QC(pt2.)
W11
AI & Physics?


CPU Components
Single Cycle CPU
Pipelined CPU

What Does Single-Cycle Mean?

- One instruction completes in one clock cycle.
  - Instruction Fetch → Decode and Register Read → Execute → Memory Access → Write Back
- Not every instruction uses every component
  - add: does not access data memory
  - sw: does not write a register
  - beq: does not access data memory or write a register
- **However, every instruction uses the same clock period**

Component Delay

- Assumed component delays

Component	Delay
Instruction Memory	200 ps
Register Read	100 ps
ALU	150 ps
Data Memory	200 ps
Register Write	100 ps

Critical Path

- Simplified model: delays of multiplexers, control logic, immediate generation, and PC update logic are omitted

Instruction	Instruction Memory	Register Read	ALU	Data Memory	Register Write	Total Delay
ADD	✓	✓	✓	—	✓	550 ps
ADDI	✓	✓	✓	—	✓	550 ps
LW	✓	✓	✓	✓	✓	750 ps
SW	✓	✓	✓	✓	—	650 ps
BEQ	✓	✓	✓	—	—	450 ps

- The clock period must be long enough for the slowest instruction

Limitations of a Single-Cycle CPU

- Clock Period ≥ 750 ps
- CPU performance
  - CPU Time = Instruction Count × CPI × Clock Cycle Time
- For a single-cycle CPU
  - CPI = 1
  - Clock Cycle Time = Limited by the critical path

Motivation for Pipelining

- Can we divide instruction execution into stages and overlap different instructions?
  - Instruction 1: IF → ID → EX → MEM → WB
  - Instruction 2: IF → ID → EX → MEM → WB
  - Instruction 3: IF → ID → EX → MEM → WB
- Pipelining Classic five stages
  - IF → ID → EX → MEM → WB

The Main Idea of Pipelining

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7
Instruction 1	IF	ID	EX	MEM	WB	—	—
Instruction 2	—	IF	ID	EX	MEM	WB	—
Instruction 3	—	—	IF	ID	EX	MEM	WB

- Pipelining does not primarily reduce the latency of one instruction
- It increases throughput: the number of completed instructions per unit time

Single-Cycle vs Pipelined

Metric	Single-Cycle CPU	Ideal 5-Stage Pipeline
Clock Period	750 ps	200 ps
Clock Frequency	Approximately 1.33 GHz	Approximately 5 GHz
Instruction Latency	750 ps	Approximately 1 ns
Steady-State Throughput	1 instruction / 750 ps	1 instruction / 200 ps

- Idealized comparison: pipeline-register overhead and hazards are omitted

Data Hazard

```asm
</> asm
add x1, x2, x3
sub x4, x1, x5
```

```asm
</> asm
lw  x1, 0(x2)
add x3, x1, x4
```

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6
`add x1, x2, x3`	IF	ID	EX	MEM	WB	
`sub x4, x1, x5`		IF	ID	EX	MEM	WB

- The second instruction needs x1 before the first instruction writes it back
- Solutions
  - Forwarding
  - Stall

Control Hazard

```asm
</> asm
beq x1, x2, target
```

- CPU may fetch instructions before the branch outcome is known
- Solutions
  - Stall
  - Flush
  - Branch Prediction

Where Modern CPUs Are Heading

- Heterogeneous Cores
  - Cores optimized for different workloads
- Specialized Accelerators
  - Dedicated hardware for specific tasks
- Chiplets and 3D Integration
  - Multiple dies combined in one package
- Pipeline Hazards
  - Reduce the cost of moving data